Natural Language Processing

Unit	Main Topics
Unit 1	NLP basics, applications, tasks, linguistics, ML/DL approaches and transformers
Unit 2	NLP workflow, data sources, preprocessing, representations and language models
Unit 3	Transformers, BERT, GPT, fine-tuning, LoRA, PEFT and LLM evaluation
Unit 4	Text generation, decoding strategies, evaluation, prompt engineering and risk mitigation
Unit 5	Cross-modal GenAI, RAG, prompt chaining, risks, safety and AI policies

Detailed 5 Unit Semester Exam Notes

UNIT 1: Basics of NLP and Deep Learning Approaches

1. Introduction to Natural Language Processing

Natural Language Processing is a branch of Artificial Intelligence that enables computers to understand, process and generate human language. Human language is difficult for machines because it is full of grammar rules, context, emotions, ambiguity and informal expressions.

NLP connects linguistics, computer science, machine learning and deep learning. Earlier NLP systems were mostly rule-based, but modern systems use statistical learning, neural networks and transformer-based models. NLP is now used in search engines, chatbots, translation systems, voice assistants and document analysis.

- It converts unstructured text or speech into useful structured information.
- It helps machines understand meaning, intent and context.
- It supports both analysis tasks and generation tasks.
- Modern NLP uses deep learning and large language models.

Application	Explanation	Example
Virtual Assistants	Understand voice or text commands and respond.	Siri, Alexa, Google Assistant
Chatbots	Answer user queries using natural language.	Customer support bot
Machine Translation	Translate text from one language to another.	English to Hindi translation
Search Engines	Understand query intent and rank documents.	Google search
Document Analysis	Extract useful information from large text.	Resume screening, legal document review

NLP contains many tasks depending on the objective. Some tasks analyze text, while some generate text. In exams, it is useful to write the definition, purpose and example of each task.

Sentiment analysis identifies emotional tone. Named Entity Recognition identifies names of people, places, organizations and dates. POS tagging assigns grammar labels to words. Summarization reduces a long document into a short meaningful form.

- **Sentiment Analysis:** classifies opinion as positive, negative or neutral.
- **NER:** detects entities such as person, location, organization, date and money.
- **POS Tagging:** labels words as noun, verb, adjective, adverb etc.
- **Summarization:** creates short summary from long text.
- **Question Answering:** finds or generates answers from text.
- **Text Classification:** assigns category such as spam, sports, politics or education.

3. Linguistic Foundations

2. Key NLP Tasks

Linguistic foundations help NLP systems understand the structure and meaning of language. Syntax deals with grammar and word order. Semantics deals with meaning. Pragmatics deals with meaning in context. Ambiguity occurs when a sentence has more than one meaning.

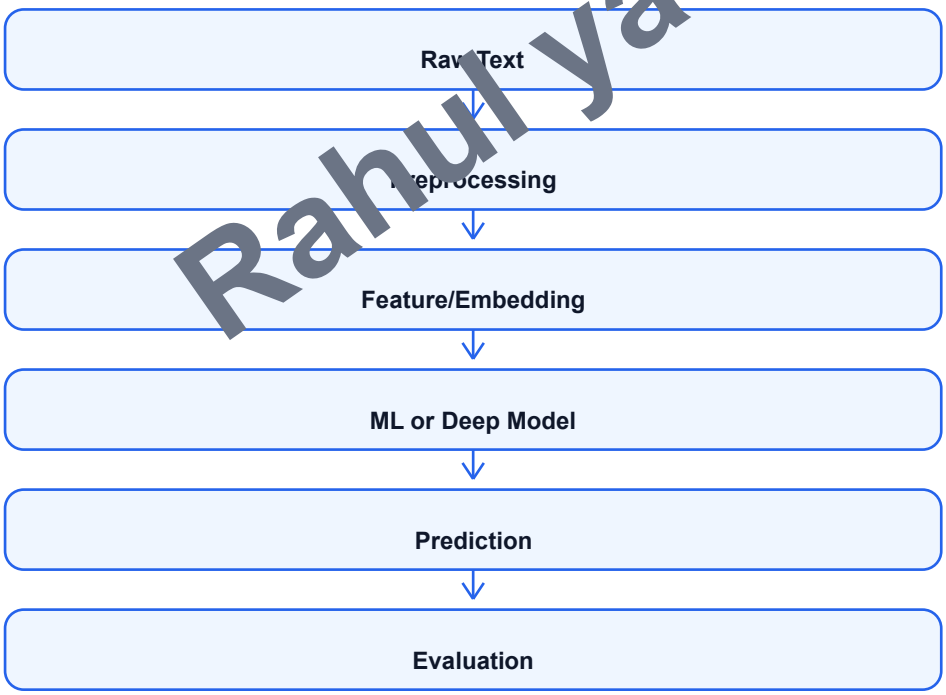
Concept	Meaning	Example
Syntax	Rules of sentence structure	Subject-verb-object order
Semantics	Meaning of words and sentences	Bank as river side or financial bank
Concept	**Meaning**	**Example**
Ambiguity	Multiple possible meanings	I saw a man with a telescope
Code-switching	Mixing two languages in one sentence	Kal class online hai
Morphology	Structure of words	play, played, playing

4. Machine Learning and Deep Learning for NLP

Machine learning approaches represent text as numerical features and then train algorithms for prediction. Traditional ML methods such as Naive Bayes and SVM perform well for simple text classification problems when features such as Bag of Words or TF-IDF are used.

Deep learning models learn representations automatically. RNNs process text sequentially, LSTMs handle long-term dependencies better, CNNs can capture local phrase patterns, and transformers use attention to understand global context. This shift from manual features to learned representations improved NLP performance.

- Naive Bayes: simple probabilistic classifier used in spam filtering.
- SVM: finds best separating boundary for text classification.
- RNN: handles sequence data but struggles with long context.
- LSTM: solves vanishing gradient problem and remembers longer context.
- CNN: captures local n-gram like patterns in text.
- Transformers: use self-attention and support large-scale language models



5. Transformers, Transfer Learning and Limitations

Transformers introduced the attention mechanism, which allows models to focus on important words regardless of their distance in the sentence. BERT is mainly used for understanding tasks because it learns bidirectional context. GPT is mainly used for generation because it predicts the next token autoregressively.

Transfer learning means using a model pretrained on a large corpus and fine-tuning it on a smaller task-specific dataset. This reduces training cost and improves performance. However, NLP models may still suffer from bias, hallucination, high computational cost, poor explainability and privacy concerns.

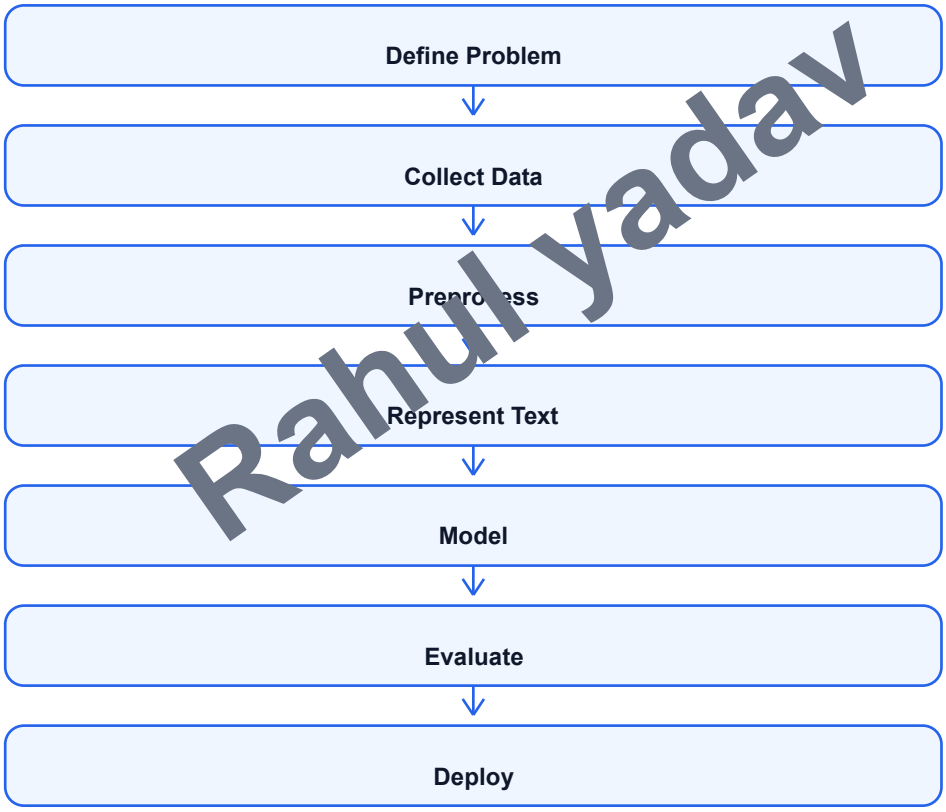
UNIT 2: NLP Project Pipeline and Data Processing

1. NLP Workflow

An NLP project should follow a proper workflow so that the final system is reliable. The workflow begins with defining the problem clearly. Then data is collected, cleaned, represented numerically, modeled, evaluated and finally deployed.

For example, if the task is sentiment analysis of product reviews, first the classes are defined as positive, negative and neutral. Then reviews are collected, cleaned, tokenized and converted into features. A model is trained and evaluated before deployment.

- Define the objective and output labels.
- Collect data from reliable sources.
- Preprocess text to remove noise.
- Convert text into numerical representation.
- Train and evaluate the model.
- Deploy the model and monitor performance.



2. Data Sources for NLP

NLP data can come from public datasets, social media, websites, documents, emails, reviews, chat logs and speech transcripts. Public datasets are useful for academic experiments because they are already labeled and widely used for benchmarking.

Web scraping is used when data is collected from websites. BeautifulSoup is a Python library used to parse HTML pages and extract useful text. Newspaper3k is used to extract article text, title, author and publication date from news websites.

- Public datasets: Kaggle, Hugging Face Datasets, UCI, GLUE, SQuAD.
- Web scraping: collect article or review data from websites.
- APIs: Twitter/X API, Reddit API, news API.
- Internal data: chat logs, customer tickets, emails, documents.
- Ethical care: respect privacy, copyright and website terms.

3. Data Preprocessing

Text data is usually noisy. It may contain punctuation, HTML tags, emojis, URLs, spelling mistakes, mixed languages and inconsistent capitalization. Preprocessing improves the quality of text before modeling.

Step	Detailed Explanation	Example
Cleaning	Remove HTML tags, URLs, punctuation, numbers or unwanted symbols.	Remove , http links
Tokenization	Split sentence into words, subwords or sentences.	NLP is useful -> NLP, is, useful
Stopword Removal	Remove common words that may not add meaning.	is, the, a, of
Normalization	Convert text into standard form.	lowercasing, spelling correction
Stemming	Cut word to root-like form.	playing -> play
Lemmatization	Convert word to dictionary form.	better -> good

4. Text Augmentation and Representation

Text augmentation increases training data by creating modified versions of text. This he[illegible]en the dataset is small. Common methods include synonym replacement, random deletion, back tran[illegible]ati[illegible]n a[illegible]d paraphrasing.

Since machine learning models cannot directly understand text, text m[illegible]t be co[illegible]verted into numerical representation. Bag of Words counts word occurrence. TF-IDF gives higher weight to imp[illegible]rtan[illegible] w[illegible]ds. Word embeddings such as Word2Vec, GloVe and FastText represent words as dense vectors.

- BoW: simple count-based representation but ignores wor[illegible] order.
- TF-IDF: highlights important words in a d[illegible]um[illegible]nt.
- GloVe: learns vectors using gl[illegible]l wor[illegible] co-occurrence.
- FastText: uses subword inform[illegible]tion and handles rare words better.

Representation	Strength	Limitation
BoW	Simple and easy for ML models	Ignores order and meaning
TF-IDF	Good for classification/search	Sparse and not contextual
Word2Vec	Captures semantic similarity	One vector per word
GloVe	Uses global statistics	Not context dependent
FastText	Handles subwords and rare words	Still not fully contextual
BERT Embedding	Context-aware representation	Computationally expensive

- Word2Vec: learns word vectors from context.

5. Contextual Embeddings and Language Models

Contextual embeddings generate different representations for the same word depending on the sentence. For example, the word 'bank' has different meanings in 'river bank' and 'bank account'. BERT and ELMo solve this problem better than traditional embeddings.

Subword models such as BPE and WordPiece divide words into smaller units. This helps handle rare words, spelling variations and multilingual text. Language models learn probability patterns of language. N-gram models are traditional models, while MLM and CLM are used in transformer systems.

- ELMo creates embeddings using bidirectional language models.

- BERT uses masked language modeling for bidirectional context.
- BPE merges frequent character pairs into subword units.
- WordPiece is used in BERT tokenization.
- N-gram LM predicts word based on previous n-1 words.
- MLM predicts masked tokens; CLM predicts next tokens.

UNIT 3: Transformers and LLM Architectures

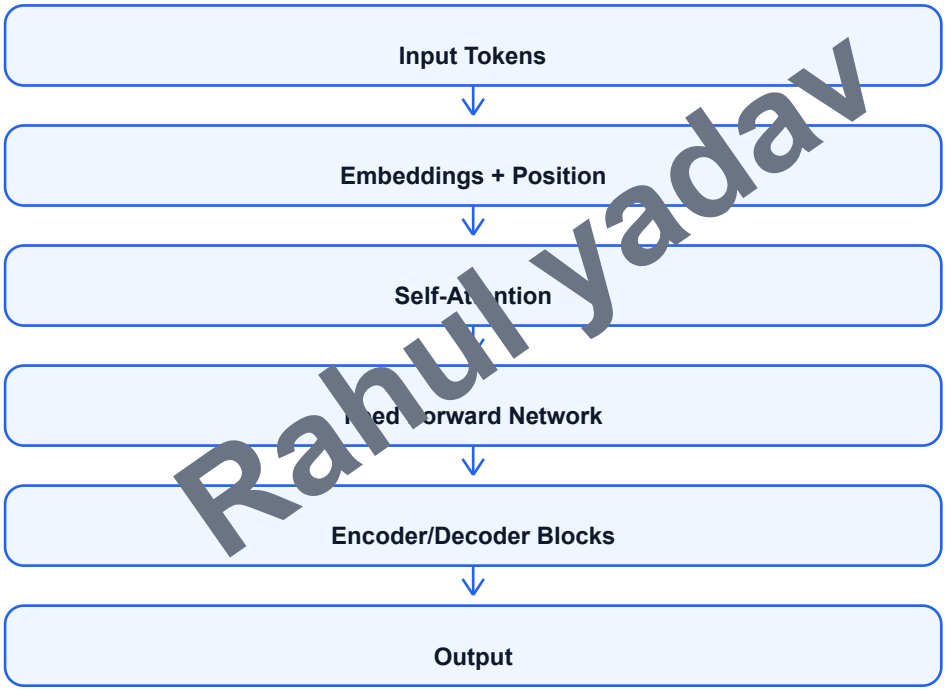
1. Transformer Core

The Transformer architecture is a deep learning architecture designed for sequence modeling. It replaced recurrent processing with attention, which allows parallel computation and better handling of long-range dependencies. Transformers are the foundation of modern models such as BERT, GPT and many LLMs.

The most important component is self-attention. Self-attention helps each token look at other tokens in the sentence and

decide which are important. Multi-head attention performs this process multiple times in parallel, allowing the model to capture different types of relationships.

- Self-attention calculates relationship between tokens.
- Multi-head attention captures different linguistic patterns.
- Encoder understands input sequence.
- Decoder generates output sequence.
- Positional encoding gives information about word order.



2. Evolution: Transformer to BERT to GPT

The original Transformer was introduced for machine translation and contained both encoder and decoder. Later, different models used parts of this architecture for different tasks. BERT uses the encoder part and is strong at understanding tasks. GPT uses the decoder part and is strong at text generation.

BERT is trained using Masked Language Modeling, where some words are hidden and the model predicts them using both left and right context. GPT is trained using Causal Language Modeling, where the model predicts the next token using previous tokens only.

- Transformer: general encoder-decoder architecture.
- BERT: encoder-only model for understanding and classification.
- GPT: decoder-only model for generation.
- BERT sees bidirectional context; GPT uses left-to-right context.

Model	Architecture	Training Objective	Best Use
Transformer	Encoder-Decoder	Sequence-to-sequence	Translation, summarization
BERT	Encoder-only	Masked Language Modeling	Classification, NER, QA
GPT	Decoder-only	Causal Language Modeling	Text generation, chat, coding

3. Fine-tuning, Pretraining, LoRA and PEFT

Large models are expensive to fine-tune completely. Parameter Efficient Fine-Tuning updates only a small number of parameters. LoRA is a popular PEFT technique that adds low-rank trainable matrices to model layers while keeping original

- LoRA is useful for adapting large language models efficiently.
- Full fine-tuning gives flexibility but needs more compute.

Evaluating LLMs is difficult because generated text can be correct in many different ways. Automatic metrics are useful but not perfect. Perplexity measures how well a model predicts text. BLEU is used for translation. ROUGE is used for summarization by comparing overlap with reference summaries.

Modern LLMs face challenges such as hallucination, bias, toxicity, privacy leakage and high computational cost. Hallucination means the model generates confident but false information. Bias means unfair patterns learned from training data.

- Perplexity: lower value generally means better language modeling.
- ROUGE: measures recall-oriented overlap for summaries.
- Human evaluation checks helpfulness, correctness and fluency.
- Challenges include hallucination, bias, compute cost and safety.

Pretraining is the process of training a model on a very large corpus to learn general language patterns. Fine-tuning adapts the pretrained model to a specific task using smaller labeled data. This approach saves time and improves performance. weights mostly frozen.

- Pretraining learns general language knowledge.
- Fine-tuning adapts model to task-specific data.
- PEFT reduces memory and training cost.

4. LLM Evaluation and Challenges

- BLEU: measures n-gram overlap for translation.

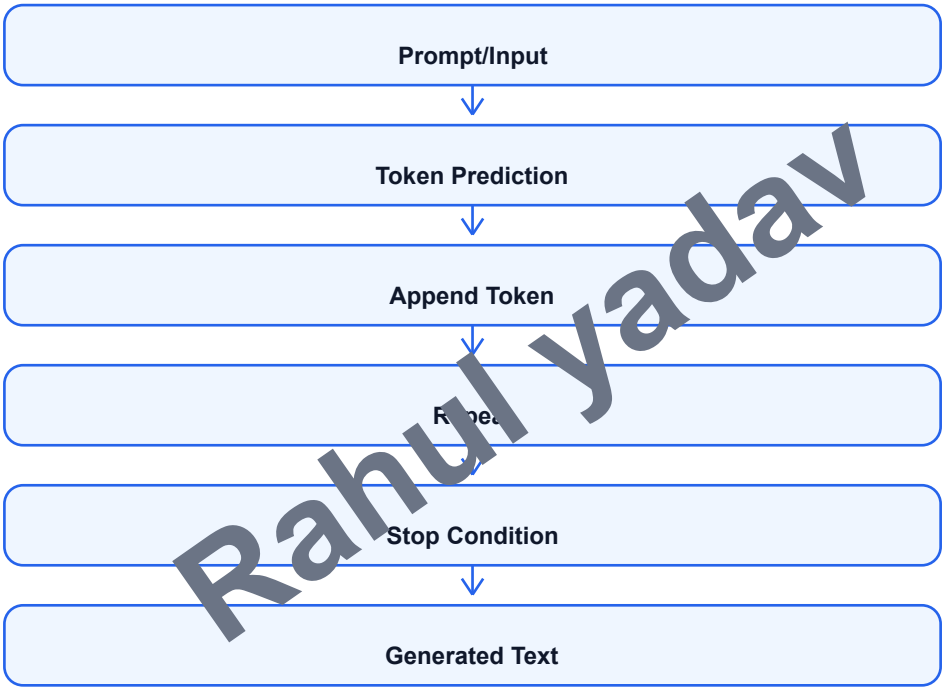
UNIT 4: Text Generation and Evaluation in Generative AI

1. Text Generation Techniques

Text generation is the process of producing new text using a language model. In autoregressive generation, the model generates one token at a time. Each new token depends on previously generated tokens. GPT-style models use this approach.

Conditional generation produces text based on a given condition such as prompt, topic, question, image or document. For example, summarization is conditioned on input document, and translation is conditioned on source language text.

- Autoregressive generation predicts next token step by step.
- Conditional generation uses an input condition or instruction.
- Applications include chatbots, summarization, translation, code generation and story writing.
- Quality depends on model, prompt, decoding strategy and training data.



2. Sampling Strategies

Decoding or sampling strategy decides how the next token is selected. Greedy decoding always selects the most probable token. It is fast but may produce repetitive or dull output. Beam search keeps multiple best candidate sequences and is useful for translation and summarization.

Top-k sampling chooses from the k most probable tokens. Top-p or nucleus sampling chooses from the smallest set of tokens whose combined probability reaches p. These methods increase diversity and creativity in generation.

- Greedy decoding: simple and deterministic but less creative.
- Beam search: considers multiple candidates but can be expensive.
- Top-k sampling: restricts choices to k likely tokens.
- Top-p sampling: dynamically selects probable token set.
- Temperature controls randomness in output.

Strategy	Working	Use
Greedy	Select highest probability token every time	Short factual output
Beam Search	Maintain multiple best sequences	Translation, summarization
Top-k	Sample from top k tokens	Creative text
Top-p	Sample from nucleus probability mass	Balanced creative generation
Temperature	Adjust randomness of probability distribution	Control diversity

Text generation evaluation is difficult because there can be many correct outputs. Automatic metrics compare generated text with reference text, but they may not capture meaning perfectly. Therefore, human assessment is also important.

Fluency checks whether text is grammatically correct and natural. BLEU is common in machine translation. ROUGE is common in summarization. Human evaluation can check helpfulness, factual correctness, coherence, safety and relevance.

- Coherence: ideas should be logically connected.

- Human assessment: best for quality and factuality.

4. Prompt Engineering, Conditioning and Safe Decoding

Prompt engineering is the process of writing effective instructions to guide a generative model. A good prompt clearly describes the role, task, format, constraints and expected output. Examples in the prompt can improve performance.

Conditioning gives additional information to the model, such as context documents, style, tone, examples or safety rules. Safe decoding includes methods that reduce harmful, biased or toxic outputs by filtering unsafe tokens or applying guardrails.

- Mention expected format such as table, bullet points or paragraph.

3. Evaluation of Generated Text

- Fluency: grammatical and readable text.
- Relevance: output should answer the prompt.

- BLEU: n-gram precision-based metric.
- ROUGE: recall overlap metric for summaries.

- Write clear task instruction.

- Provide context and examples when needed.
- Use constraints to avoid irrelevant output.
- Apply filters and guardrails for safety.

5. Mitigating Risks

Generative AI can produce biased, false or harmful text. Hallucination occurs when a model generates information that sounds correct but is not factually true. Toxicity occurs when the output contains abusive, hateful or unsafe content.

Risk mitigation includes better data filtering, RLHF, guardrails, retrieval augmentation, human review and policy-based refusal for harmful requests. For important domains such as healthcare, law and finance, generated output should be verified by experts.

- Use retrieval or trusted sources for factual tasks.
- Use toxicity filters and safety classifiers.
- Apply bias testing on model outputs.
- Use human review for high-risk decisions.
- Clearly mention limitations of generated content.

UNIT 5: Cross-Modal GenAI and Safety in LLMs

1. Cross-Modal Generative AI

Cross-modal Generative AI refers to systems that can work across multiple data types such as text, image, audio, video and code. These systems can convert one modality into another, such as text-to-image, text-to-code or text-to-video.

Text-to-image models generate images from natural language prompts. DALL-E and Stable Diffusion are popular examples. Text-to-code models generate code from instructions and help developers write, debug and explain programs.

- Text-to-image: prompt is converted into an image.
- Text-to-code: natural language is converted into program code.
- Audio generation: produces speech, music or sound.
- Video generation: creates moving visual content from prompt.
- Multimodal models combine text, images and other media.

System	Modality	Use
DALL-E	Text to Image	Generate images from prompts
Stable Diffusion	Text to Image	Open image generation and editing
Codex	Text to Code	Generate and explain code
Gemini	Multimodal	Text, image, code and reasoning tasks
Audio Models	Text to Audio	Speech, voice and music generation

Prompt chaining breaks a complex task into smaller prompts. Output of one step becomes input to the next step. This improves control, reasoning and reliability. For example, one prompt extracts facts, the next summarizes them, and the third

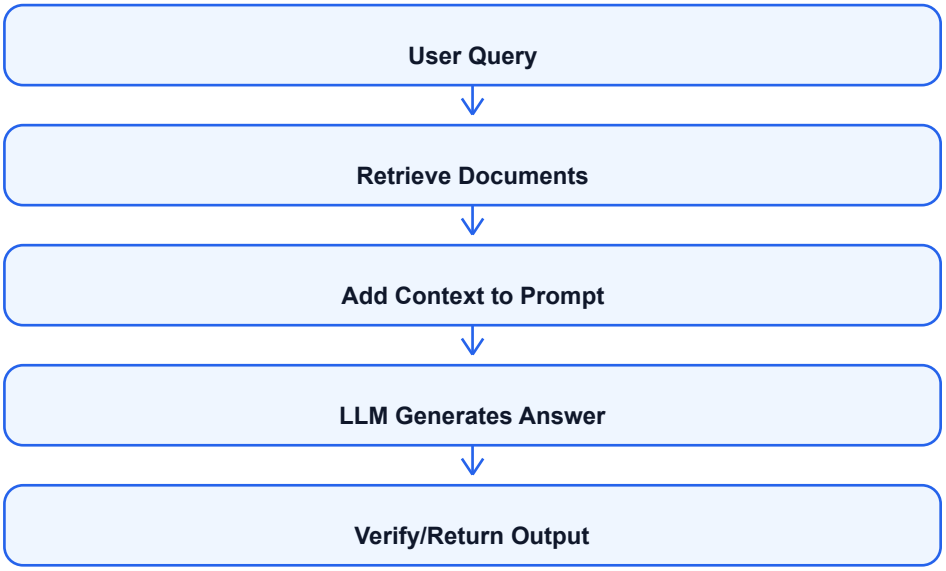
Retrieval Augmented Generation combines search with generation. Instead of depending only on model memory, the system retrieves relevant documents and gives them to the model as context. RAG reduces hallucination and helps answer

- Prompt chaining improves step-by-step task execution.
- RAG retrieves external documents before generation.
- RAG is useful for private knowledge bases and updated information.
- Good retrieval quality is important for good answers.

2. Prompt Chaining and RAG

formats the answer. domain-

specific questions. responses.



3. Risks of Cross-Modal GenAI

Cross-modal GenAI creates powerful opportunities but also serious risks. Deepfakes can create realistic fake images, videos or voices of real people. Synthetic media can spread misinformation, damage reputation or manipulate public opinion.

Text-to-code systems can generate insecure code if not reviewed. Image and audio models may reproduce bias from training data. Misuse can include fake news, impersonation, plagiarism, fraud and harmful automation.

- Deepfakes and impersonation.
- Synthetic misinformation and fake evidence.
- Copyright and ownership issues.
- Biased or harmful generated content.
- Insecure generated code or misuse of automation.

4. Safety Measures: RLHF, Guardrails and Filtering

Safety in LLMs means reducing harmful behavior and making systems more reliable. RLHF stands for Reinforcement Learning from Human Feedback. In RLHF, humans rank outputs and the model is trained to prefer helpful, honest and safe

Guardrails are rules or systems that restrict model behavior. Filtering removes unsafe input or output. Safety classifiers can detect toxicity, violence, hate, self-harm or privacy leakage. Human-in-the-loop review is important for high-risk applications.

- RLHF aligns model output with human preferences.
- Guardrails restrict unsafe behavior.
- Input filters detect harmful prompts.
- Output filters block toxic or private content.

- Human review is needed for sensitive domains.

5. Policies: EU AI Act and NITI Aayog Guidelines

AI policies aim to ensure safe, transparent and responsible use of AI. The EU AI Act follows a risk-based approach. It classifies AI systems into categories such as unacceptable risk, high risk and limited risk. High-risk systems need stronger transparency, documentation and safety checks.

NITI Aayog has promoted responsible AI principles in India. Important principles include safety, reliability, fairness, inclusiveness, transparency, accountability and privacy. These guidelines encourage AI systems that benefit society while reducing harm.

- EU AI Act uses risk-based regulation.
- High-risk AI systems require strict compliance.
- Responsible AI promotes fairness and accountability.
- Privacy and transparency are important policy goals.
- AI developers should document limitations and risks.

Important Semester Exam Questions

Unit 1

- Define NLP and explain its applications.
- Explain key NLP tasks with examples.
- Explain syntax, semantics, ambiguity and code-switching.
- Compare ML and deep learning approaches for NLP.
- Explain transformers, BERT and GPT in brief.

Unit 2

- Compare BoW, TF-IDF, Word2Vec, GloVe and FastText.
- Explain contextual embeddings and subword models.
- Differentiate N-gram, MLM and CLM language models.
- Explain NLP project pipeline with diagram.
- Explain text preprocessing techniques.

Unit 3

- Explain pretraining, fine-tuning, LoRA and PEFT.
- Explain self-attention and multi-head attention.
- Compare BERT and GPT architectures.
- Explain autoregressive and conditional text generation.
- Compare greedy, beam, top-k and top-p decoding.
- Explain LLM evaluation metrics.
- Write challenges of LLMs.
- How can bias, hallucination and toxicity be reduced?

Unit 4

- Explain cross-modal Generative AI with examples.
- Explain BLEU, ROUGE and human evaluation.
- Explain prompt engineering and safe decoding.

Unit 5

- Explain text-to-image and text-to-code systems.
- Explain RAG and prompt chaining.
- Write risks of deepfakes and synthetic media.

- Explain RLHF, guardrails, filtering and AI policies.